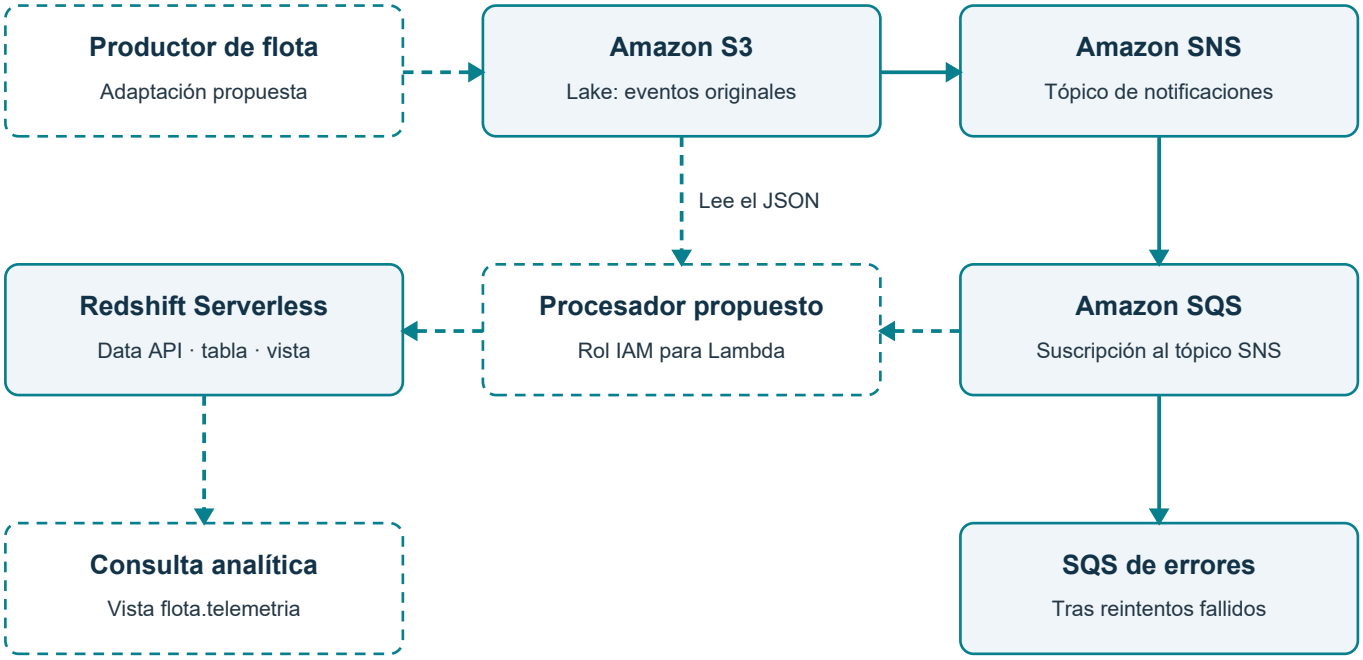
Pipeline de telemetría vehicular en AWS

Integrantes: Sebastián Giraldo Franco y Juan José Jaramillo Mora.
Nube elegida: Amazon Web Services (AWS).

1. Diagrama de la arquitectura

Diseñamos un pipeline para conservar y preparar la telemetría de una flota simulada: identificador del vehículo, velocidad, combustible y ubicación. El flujo separa los datos originales, las notificaciones de llegada, el procesamiento y la consulta histórica.



Cómo se interpreta: el productor guarda el evento en S3. La notificación de creación pasa por SNS y su suscripción SQS. El procesador consume la referencia, lee el lake y escribe en Redshift. La cola de errores recibe mensajes que no pudieron procesarse tras los reintentos.

Los elementos discontinuos representan ejecución propuesta. En Terraform se modelan la identidad y los permisos del procesador; la adaptación del productor y la función Lambda no están desarrolladas. El diagrama describe la arquitectura diseñada, sin afirmar que esté desplegada.

2. Justificación de cada pieza

Pieza elegida	Papel en el pipeline y razón de elección
Amazon S3	Conserva el JSON original como data lake. Permite reprocesar eventos sin depender de las tablas analíticas. Se eligió almacenamiento de objetos porque los datos de origen son archivos y deben preservarse antes de transformarlos; el bucket es privado, cifrado y versionado.
Amazon SNS estándar	Publica avisos cuando se crea un objeto en el lake. Separa al productor de los consumidores y permite agregar nuevas suscripciones. Se usa un tópico estándar, compatible con las notificaciones de S3.
Amazon SQS	Su suscripción al tópico conserva las referencias pendientes y amortigua picos. El procesamiento puede reintentarse sin que el productor deba esperar. Una segunda cola, la DLQ, aísla mensajes que fallan repetidamente.
Rol IAM del procesador	Representa la identidad del job que leería SQS y S3 y cargaría Redshift. Lambda es la ejecución propuesta por su integración con SQS. El rol separa los accesos necesarios para procesar datos de los permisos de administración.
Redshift Serverless	Es el warehouse para consultas SQL históricas. La tabla flota.eventos_telemetria tiene esquema definido, y la vista flota.telemetria presenta un evento por UUID. Data API permite enviar las inserciones sin abrir una conexión SQL directa desde el job.
VPC y subredes privadas	Aíslan el workgroup de Redshift del acceso público. El diseño usa tres subredes y un grupo de seguridad sin ingreso SQL; el acceso previsto es mediante la API administrada.

3. Conexiones y recorrido de un dato

1. El productor adaptado generaría una lectura de V03 y añadiría un UUID y fecha UTC. Guardaría el JSON en **raw/flota/año/mes/día/UUID.json**; los reintentos conservarían el mismo identificador.

2. S3 detecta la creación del archivo y publica en SNS la referencia al bucket y a la clave. SNS entrega ese aviso a SQS mediante la suscripción, sin la envoltura adicional de SNS. El mensaje anuncia dónde está el dato; el dato original permanece en el lake.

3. El procesador propuesto recibiría el aviso, leería el JSON de S3 y validaría campos, tipos y rangos. Añadiría la fecha de ingesta e insertaría la lectura en Redshift usando Data API. Confirmaría el mensaje solo cuando la ejecución SQL termine correctamente.

4. El analista consultaría la vista **flota.telemetria**. Esta conserva una fila por UUID mediante ROW_NUMBER, priorizando la primera fecha de ingesta. Así los reintentos no se cuentan varias veces en las consultas, aunque la tabla física pueda conservar duplicados.

4. Permisos y principio de mínimo privilegio

Mínimo privilegio significa conceder a cada identidad únicamente las acciones sobre los recursos que necesita para cumplir su función. Lo aplicamos separando al productor, al procesador y a la identidad de administración: escribir originales, transformarlos y administrar infraestructura son responsabilidades diferentes.

Acceso del procesador	Permiso y alcance	Por qué se necesita
Consumir la ingesta	sqs:ReceiveMessage, sqs:DeleteMessage y sqs:GetQueueAttributes, solo en la cola principal.	Recibir trabajo y confirmar lo que termina correctamente. No necesita crear, modificar ni eliminar colas.
Leer el lake	s3:GetObject, únicamente sobre raw/flota/* del bucket.	La notificación proporciona la clave exacta. No necesita listar el bucket, escribir ni borrar los originales.
Enviar SQL	redshift-data:ExecuteStatement y redshift-serverless:GetCredentials, limitados al workgroup del proyecto.	Cargar datos con credenciales temporales. El acceso a la API se combina con permisos SQL específicos.
Verificar el resultado	redshift-data:DescribeStatement con Resource: * y condición de propietario de la sentencia.	La ejecución es asíncrona: hay que comprobar que terminó. Esta acción no admite un ARN individual; la condición restringe su consulta al ejecutor.
Escribir en la tabla	USAGE en el esquema flota e INSERT en flota.eventos_telemetria para el usuario SQL IAMR del rol.	Permite cargar telemetría sin ser administrador ni propietario. No se conceden SELECT, UPDATE, DELETE o DROP sobre la tabla.

El rol confía en el servicio Lambda para que el job futuro pueda asumirlo. Su usuario SQL tiene la contraseña deshabilitada y utiliza autenticación temporal. También se revocan los permisos heredados de creación en el esquema public y de tablas temporales en la base dedicada.

Permisos que conectan los servicios. El tópico autoriza a S3 a publicar solo desde el bucket y la cuenta previstos. La cola autoriza a SNS a entregar mensajes solo desde el tópico y la cuenta del proyecto. Estas políticas impiden que la relación entre servicios se abra a orígenes no previstos.

Separación del productor y la administración. La política del productor solo permite PutObject en raw/flota/* y se adjuntaría a su identidad al implementar la adaptación. Terraform usaría el secreto administrativo gestionado por Redshift para crear el esquema y conceder permisos; ese secreto no se entrega al procesador. Los accesos de un analista serían independientes y de solo lectura sobre la vista.

5. Decisión de diseño y alternativa descartada

Decisión: SNS con una suscripción SQS para la ingesta, en lugar de conservar Kafka mediante Amazon MSK. El productor del trabajo anterior envía a Kafka local. Consideramos mantener ese protocolo con Kafka administrado en AWS, lo que permitiría reutilizar su mecanismo de envío.

Descartamos MSK para este alcance porque añadiría configuración de conectividad, autenticación y operación de Kafka que no resulta necesaria para el objetivo del trabajo. Elegimos conservar la simulación de vehículos y proponer su adaptación a S3: las notificaciones SNS y la cola SQS desacoplan la llegada de datos del procesamiento y expresan el patrón de tópico y suscripción requerido.

La decisión tiene un costo de adaptación: el productor actual no puede usarse sin cambios. También se pierde el modelo de consumo por offsets de Kafka; el reproceso se basaría en los originales de S3. SNS aporta la posibilidad de añadir consumidores mediante suscripciones independientes, mientras SQS mantiene el trabajo pendiente de cada consumidor.

6. Escalabilidad ante 100 veces más datos

El simulador emite aproximadamente una lectura por segundo de cada uno de ocho vehículos al inicio: unos **8 eventos/s**. Cien veces ese volumen equivale a **800 eventos/s**, suponiendo el mismo patrón. Esta cifra es una estimación para discutir el diseño, no un resultado de pruebas de rendimiento.

Pieza	Qué podría mantenerse y qué habría que ajustar
S3	Se mantiene como almacenamiento desacoplado y permite crecer sin cambiar de modelo. Sin embargo, un JSON por lectura multiplicaría objetos pequeños: habría que agrupar eventos y evaluar archivos columnares como Parquet para los lotes analíticos, preservando el dato original según la política de conservación.
SNS y SQS	Mantienen el desacoplamiento y absorben picos mientras el consumidor se recupera. Habría que revisar cuotas, antigüedad y cantidad de mensajes pendientes. La retención de cuatro días no reemplaza capacidad de procesamiento suficiente; una cola que crece continuamente indica atraso.
Procesador	Sería uno de los primeros puntos a ajustar: tamaño de lote, concurrencia y duración por ejecución. Requeriría manejo de fallos parciales y reintentos seguros. Incrementar consumidores sin control podría trasladar el cuello de botella al warehouse.
Redshift	La configuración fija capacidad base y máxima de 8 RPU. Se medirían tiempos de carga y consulta antes de aumentar esa capacidad. Para volumen alto se evaluaría COPY por lotes desde S3 en lugar de muchas inserciones pequeñas, con un rol de lectura para Redshift y permisos acotados adicionales.

La vista actual elimina repeticiones en las consultas, pero no reduce duplicados físicos. Si el volumen vuelve costosa esa operación, se evaluaría una carga con staging y MERGE. La prioridad sería medir retrasos y saturación del procesador y del warehouse; la capacidad de los servicios administrados no garantiza por sí sola el rendimiento de todo el pipeline.